

RESTFul APIs - Week 3

Summary

1. What is REST?
2. Using HTTP Methods for RESTful Services
3. Resource Naming
4. Idempotence of REST APIs
5. HTTP Status Codes
6. Homework (due in **fourth** week)

What is REST?

REST (Representational State Transfer) is an architectural style that has six main rules that should be followed in order to design a true RESTful API:

- *Uniform Interface* - a communication interface should be defined between clients and servers ; this simplifies and decouples the architecture, which enables each part to evolve independently;
 - *Resource-Based*
 - URIs are used as resource identifiers;
 - resources are conceptually separate from the representations that are returned to the client (the server does not send its database, but rather, some HTML, XML or JSON).
 - *Manipulation of Resources Through Representations*
 - client holds a representation of a resource and has enough information to modify or delete the resource on the server.
 - *Self-descriptive Messages*
 - each message includes enough information to describe how to be processed;
 - *Hypermedia as the Engine of Application State (HATEOAS)*
 - clients deliver state via body contents, query-string parameters, request headers and the requested URI (the resource name). Services deliver state to clients via body content, response codes, and response headers. This is technically referred-to as hypermedia (or hyperlinks within hypertext).

- *Stateless*
 - the necessary state to handle the request is contained within the request itself, whether as part of the URI, query-string parameters, body, or headers.
 - the URI uniquely identifies the resource and the body contains the state (or state change) of that resource;
 - after the server does the processing, the appropriate state is communicated back to the client via headers, status and response body;
 - in REST, the client must include in the request all information needed by the server to fulfill it;
 - statelessness enables greater scalability since the server does not have to maintain, update or communicate that session state;
 - **State, or application state** - what the server needs to fulfill a request—data necessary for the current session or request; could vary by client and per request;
 - **A resource, or resource state** - the data that defines the resource representation—the data stored in the database, for instance; constant across every client who requests it;

- *Cacheable* - clients can cache responses; Responses must define themselves as cacheable, or not, to prevent clients reusing stale or inappropriate data in response to further requests;

- *Client-Server*
 - the uniform interface separates clients from servers;
 - clients are not concerned with data storage, which remains internal to each server, so that the portability of client code is improved;
 - servers are not concerned with the user interface or user state, so that servers can be simpler and more scalable;
 - servers and clients may also be replaced and developed independently, as long as the interface is not altered.

- *Layered System* - a client cannot tell whether it is connected directly to the end server, or to an intermediary along the way; intermediary servers may improve system scalability by enabling load-balancing and by providing shared caches. Layers may also enforce security policies.

- *Code on Demand* (optional) - Servers are able to temporarily extend or customize the functionality of a client by transferring logic to it that it can

execute. Examples of this may include compiled components such as Java applets and client-side scripts such as JavaScript.

Resource: <https://www.restapitutorial.com/lessons/whatisrest.html>

Using HTTP Methods for RESTful Services

The primary or most-commonly-used HTTP verbs (or methods, as they are properly called) are POST, GET, PUT, PATCH, and DELETE. These correspond to create, read, update, and delete (or CRUD) operations, respectively. There are a number of other verbs, too, but are used less frequently. Of those less-frequent methods, OPTIONS and HEAD are used more often than others.

HTTP Verb	CRUD	Entire Collection (e.g. /customers)	Specific Item (e.g. /customers/{id})
POST	Create	201 (Created), 'Location' header with link to /customers/{id} containing new ID.	404 (Not Found), 409 (Conflict) if resource already exists..
GET	Read	200 (OK), list of customers. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single customer. 404 (Not Found), if ID not found or invalid.
PUT	Update/Replace	405 (Method Not Allowed), unless you want to update/replace every resource in the entire collection.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
PATCH	Update/Modify	405 (Method Not Allowed), unless you want to modify the collection itself.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection—not often desirable.	200 (OK). 404 (Not Found), if ID not found or invalid.

Resource: <https://www.restapitutorial.com/lessons/httpmethods.html>

Resource Naming

- primary data representation is called Resource - having a strong and consistent REST resource naming strategy – will definitely prove your one of the best design decisions in long term.
- a RESTful API ends up being simply a collection of URIs, HTTP calls to those URIs and some JSON and/or XML representations of resources, many of which will contain relational links.
- RESTful URI should refer to a resource that is a thing (noun) instead of referring to an action (verb)
- the RESTful principal of addressability is covered by the URIs - each resource has its own address or URI—every interesting piece of information the server can provide is exposed as a resource

Some example resources are:

- Users of the system.
- Courses in which a student is enrolled.
- A user's timeline of posts.
- The users that follow another user.
- An article about horseback riding.

Each resource in a service suite will have at least one URI identifying it. And it's best when that URI makes sense and adequately describes the resource. URIs should follow a predictable, hierarchical structure to enhance understandability and, therefore, usability: predictable in the sense that they're consistent, hierarchical in the sense that data has structure—relationships. This is not a REST rule or constraint, but it enhances the API.

RESTful APIs are written for consumers. The name and structure of URIs should convey meaning to those consumers.

Resource: <https://www.restapitutorial.com/lessons/restfulresourcenaming.html>

Idempotence of REST APIs

- an operation (or service call) is idempotent if clients can make that same call repeatedly while producing the same result;
- idempotent operations produce the same result on the server (no side effects), but the response may not be the same (e.g. a resource's state may change between requests);
- The PUT and DELETE methods are defined to be idempotent;

- DELETE would normally return a 200 (OK) or 204 (No Content) if successful, but more often it returns a 404 (Not Found) on subsequent calls, unless the service is configured to "mark" resources for deletion without actually deleting them; the state on the server is the same after each DELETE call, but the response is different; therefore it is idempotent;
- GET, HEAD, OPTIONS and TRACE methods are defined as safe, meaning they are only intended for retrieving data. This makes them idempotent as well since multiple, identical requests will behave the same.
- POST and PATCH are neither safe, nor idempotent;

Resource: <https://www.restapitutorial.com/lessons/idempotency.html>

HTTP Status Codes

Status codes are issued by a server in response to a client's request made to the server. All HTTP response status codes are separated into five classes (or categories). The first digit of the status code defines the class of response.

Status Class	Description	Examples
1xx	Informational - The request was received, continuing process	100 Continue
2xx	Successful - The request was successfully received, understood, and accepted	200 OK 201 Created 204 No Content
3xx	Redirection - Further action needs to be taken in order to complete the request	301 Moved Permanently 304 Not Modified
4xx	Client Error - The request contains bad syntax or cannot be fulfilled	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 409 Conflict
5xx	Server Error - The server failed to fulfill an apparently valid request	500 Internal Server Error 501 Not Implemented 503 Service Unavailable

Homework 2 (due in fourth week)

Create an application that provides a RESTful API.

Observation: Is mandatory to use at least: GET, POST, PUT, DELETE. It is very important to respect all additional requirements specified in the laboratory.

Additional Information:

- You are **not** allowed to use frameworks for this homework.
- Your API should communicate using either JSON or XML.
- You should provide two routes for each HTTP verb (ex. GET can be applied either on a resource or on a collection of resources - please refer to the table in section [Using HTTP Methods for RESTful Services](#)).
- Its uniform interface should be “Resource-based” (refer to [What is REST?](#) section).
- Make sure you design your API to be intuitive and easy to use by naming your resources in a logical manner (refer to [Resource Naming](#) section)
- Make sure to respect the idempotence and safeness properties for the HTTP verbs that have one or both (refer to [Idempotence of REST APIs](#))
- Make sure your API responds with the corresponding status codes for each situation it reaches in the processing of a request (ex. 200 - Ok, 404 - Not Found, 500 - Internal Server Error etc.)
- For storing the resources, you must use a persistent storage (XML file or any database of your choice)
- You have to create a POSTMAN (<https://www.getpostman.com/>) collection of requests for each call that can be made in your API, export it and commit it on your repository alongside with your code. **Students that have an API without a prepared collection of requests will NOT be evaluated!**

General observations:

- The homework can be implemented in any programming language.
- This homework should be solved individually by each of you.
- This homework, like the previous one, shall be presented using the video projector, so make sure to bring all the tools/adapters you may need.
- The complexity of the API will also be an evaluation criteria in the final homework grade.